

Unit 7- Collaborative Discussion 3 – Secure Coding Practices

Initial Post

The original authentication system contains several security weaknesses that should be addressed before it can be considered suitable for real-world use. The identified weaknesses are classified using the MITRE Common Weakness Enumeration (CWE) taxonomy, while the refactoring measures are informed by OWASP secure coding guidance (MITRE, 2026; OWASP Foundation, n.d.).

CWE-256: Plaintext Storage of a Password

Passwords are stored directly in plaintext, so anyone who gains access to the stored data could read them. In the refactored version, passwords are salted and hashed using `bcrypt`, and `bcrypt.checkpw()` is used during authentication

CWE-89: SQL Injection

The task identifies `admin' OR '1'='1` as an SQL injection or logic flaw. However, the supplied Python code does not construct or execute an SQL query, so this input would not bypass authentication as written. If database functionality were introduced later, parameterised queries should be used to prevent SQL injection.

CWE-521: Weak Password Requirements

The original system accepts weak passwords such as admin123 and password, increasing the likelihood of password guessing. The refactored version improves the password policy by enforcing a minimum password length of 12 characters during registration.

CWE-20: Improper Input Validation

The original code does not validate username or password inputs. The refactored version checks input types, strips leading and trailing spaces from usernames, restricts usernames to an approved format, and applies password validation during registration.

CWE-307: Improper Restriction of Excessive Authentication Attempts

The original system allows unlimited login attempts. The refactored version tracks failed attempts and temporarily locks the account after five unsuccessful authentication attempts.

Application Logic Flaw: Duplicate Username Registration

The original system does not enforce username uniqueness, allowing multiple user objects to be registered with the same username. The refactored version checks whether the username already exists and rejects duplicate registrations before creating a new user.

Refactored Code

```
import re
import time
import bcrypt

# Maximum number of failed login attempts before temporary lockout
MAX_ATTEMPTS = 5

# Lockout duration in seconds
```

```

LOCKOUT_SECONDS = 60

# Username validation:
# 3 to 50 characters
# Letters, numbers, underscore, dot and hyphen only
USERNAME_PATTERN = re.compile(r"^[A-Za-z0-9_.-]{3,50}$")

class User:
    # Represent a user while keeping the password hash internal

    def __init__(self, username: str, password_hash: bytes) -> None:
        self.username = username

        # Store only the password hash, not the plaintext password
        self._password_hash = password_hash

    def verify_password(self, password: str) -> bool:
        # Check the supplied password against the stored bcrypt hash

        return bcrypt.checkpw(
            password.encode("utf-8"),
            self._password_hash
        )

class AuthenticationSystem:
    # Manage user registration, authentication and simple login throttling

    def __init__(self) -> None:

        # Store username
        self.users: dict[str, User] = {}

        # Track failed login attempts for each username
        self.failed_attempts: dict[str, int] = {}

        # Store the time at which each temporary lockout expires
        self.lockout_until: dict[str, float] = {}

    def _validate_username(self, username: str) -> str:
        # Validate the username before it is used

        # Reject non string usernames
        if not isinstance(username, str):
            raise ValueError("Invalid username.")

```

```

    # Remove leading and trailing spaces
    username = username.strip()

    # Reject usernames that do not match the allowed format
    if not USERNAME_PATTERN.fullmatch(username):
        raise ValueError("Invalid username.")

    return username

def _validate_password(self, password: str) -> None:

    # The password must be a string and contain at least 12 characters
    if not isinstance(password, str) or len(password) < 12:
        raise ValueError(
            "Password must contain at least 12 characters."
        )

def add_user(self, username: str, password: str) -> None:

    # Validate registration input
    username = self._validate_username(username)
    self._validate_password(password)

    # Prevent an existing username from being overwritten
    if username in self.users:
        raise ValueError("User already exists.")

    # Hash and salt the password using bcrypt
    password_hash = bcrypt.hashpw(
        password.encode("utf-8"),
        bcrypt.gensalt()
    )

    # Store the new User object
    self.users[username] = User(
        username,
        password_hash
    )

def _register_failure(
    self,
    username: str,
    now: float
) -> None:

```

```

# Track consecutive authentication failures and apply a temporary lockout

# Increase the failed attempt counter
count = self.failed_attempts.get(
    username,
    0
) + 1

# Lock the account when the limit is reached
if count >= MAX_ATTEMPTS:

    self.lockout_until[username] = (
        now + LOCKOUT_SECONDS
    )

    # Reset the counter when the lockout begins
    self.failed_attempts[username] = 0

else:
    self.failed_attempts[username] = count

def authenticate(
    self,
    username: str,
    password: str
) -> bool:
    # Authenticate a user without revealing whether the username or password
    was incorrect

    # Validate the username
    try:
        username = self._validate_username(username)

    except ValueError:
        return False

    # Reject non string passwords
    if not isinstance(password, str):
        return False

    now = time.time()

    # Check whether the account is temporarily locked
    if now < self.lockout_until.get(username, 0):
        return False

```

```

        # Find the user
        user = self.users.get(username)

        # Unknown usernames fail safely
        if user is None:
            return False

        # Verify the supplied password against the bcrypt hash
        if user.verify_password(password):

            # Reset failed attempts after successful authentication
            self.failed_attempts[username] = 0

            # Remove any previous lockout entry
            self.lockout_until.pop(username, None)

            return True

        # Record an unsuccessful authentication attempt
        self._register_failure(
            username,
            now
        )

        return False

# Testing

if __name__ == "__main__":

    auth = AuthenticationSystem()

    # Test 1:
    # Register a new user

    auth.add_user(
        "admin",
        "Password123!"
    )

    print("First registration successful.")
    print(
        "Number of users:",
        len(auth.users)
    )

```

```

)

# Test 2:
# Try to register the SAME username again

try:
    auth.add_user(
        "admin",
        "NewPassword456!"
    )

    print("Second registration successful.")

except ValueError as error:
    print(
        "Second registration rejected:",
        error
    )

# Confirm that only one user exists
print(
    "Number of users after second attempt:",
    len(auth.users)
)

# Test 3:
# Authenticate using the correct password

print(
    "Correct password:",
    auth.authenticate(
        "admin",
        "Password123!"
    )
)

# Test 4:
# Authenticate using an incorrect password
print(
    "Incorrect password:",
    auth.authenticate(
        "admin",
        "WrongPassword123!"
    )
)

```

```

)
# Test 5:
# Reset authentication state and test account logout
auth.failed_attempts["admin"] = 0
auth.lockout_until.pop("admin", None)

print("\nTesting account logout...")

for attempt in range(1, MAX_ATTEMPTS + 1):

    result = auth.authenticate(
        "admin",
        "WrongPassword123!"
    )

    print(
        f"Failed attempt {attempt}:",
        result
    )

# Test 6:
# Try the correct password while the account is locked
result = auth.authenticate(
    "admin",
    "Password123!"
)

print(
    "\nCorrect password during logout:",
    result
)

# Test 7:
# Wait for the lockout period to expire and try the correct password after
the lockout expires
print(
    f"\nWaiting {LOCKOUT_SECONDS} seconds "
    "for the lockout to expire..."
)

time.sleep(LOCKOUT_SECONDS + 1)

# Try the correct password after the lockout expires
result = auth.authenticate(
    "admin",

```

```
        "Password123!"
    )
    print(
        "Correct password after logout:",
        result
    )
```

execution result.

The refactored authentication system was tested to verify the implemented security controls. The following tests were performed:

1. **User registration:** A new user was registered successfully.
2. **Duplicate username registration:** A second registration using the same username was rejected.
3. **Correct password authentication:** Authentication returned True.
4. **Incorrect password authentication:** Authentication returned False.
5. **Account logout:** Five consecutive failed login attempts triggered the temporary logout mechanism.
6. **Authentication during logout:** The correct password was rejected while the account was locked.
7. **Authentication after logout expiry:** After the 60-second logout period expired, the correct password was accepted successfully.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\code> python3.exe .\Authentication_System.py
First registration successful.
Number of users: 1
Second registration rejected: User already exists.
Number of users after second attempt: 1
Correct password: True
Incorrect password: False

Testing account logout...
Failed attempt 1: False
Failed attempt 2: False
Failed attempt 3: False
Failed attempt 4: False
Failed attempt 5: False

Correct password during logout: False

Waiting 60 seconds for the logout to expire...
Correct password after logout: True
PS C:\code> █
```

References

MITRE. (2026) *CWE – Common Weakness Enumeration*. Available at: <https://cwe.mitre.org/> (Accessed: 6 September 2026).

OWASP Foundation. (no date) *Secure Coding Practices – Quick Reference Guide*. Available at: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/> (Accessed: 6 September 2026).

OWASP Foundation. (no date) *Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Accessed: 5 September 2026).

OWASP Foundation. (no date) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 5 September 2026).